

web_scraping Group5

June 28, 2026

Use Python, `requests`, `BeautifulSoup` and/or `pandas` to scrape web data:

0.1 Import Libraries

Wir laden hier Beautiful Soup und Pandas, die wir später brauchen.

```
[2]: import time
      from urllib.parse import urljoin

      import pandas as pd
      import requests
      from bs4 import BeautifulSoup
```

0.2 Define the Target URL - Vienna City Marathon

Wir wollen alle Ergebnisse des Vienna City Marathon scrapen. Um den Server nicht zu überlasten, setzen wir einen Delay von 1 Sekunde zwischen die Abfragen ein.

```
[3]: url = "https://vienna.r.mikatiming.de/2026/"
      BASE_DOMAIN = "https://vienna.r.mikatiming.de"
      REQUEST_DELAY_SECONDS = 1.0
```

Für diese Demonstration scrapen wir die Jahre 2017 bis 2026. Die Requests bleiben aus ethischen Gründen auf mindestens 1 Sekunde Abstand begrenzt.

```
[4]: START_YEAR = 2017
      END_YEAR = 2026
```

0.3 Send a Request to the Website

Wir erstellen eine Session mit Headern und laden die Startseite testweise.

```
[5]: session = requests.Session()
      session.headers.update({
          "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36_
↵
          "(KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36",
          "Accept-Language": "en-US,en;q=0.9",
      })
```

```

response = session.get(url, timeout=30)
print("Status Code:", response.status_code)
response.raise_for_status()

html = response.text
print("Downloaded HTML chars:", len(html))

```

Status Code: 200

Downloaded HTML chars: 46102

0.4 Parse the HTML Content

Wir parsen das HTML mit BeautifulSoup und prüfen kurz, ob statische Tabellen vorhanden sind.

```

[6]: soup = BeautifulSoup(html, "html.parser")
print("Page title:", soup.title.get_text(strip=True) if soup.title else "No_
    ↳title found")

print("Static tables found in HTML:", len(soup.find_all("table")))

```

Page title: 43. Vienna City Marathon 2026

Static tables found in HTML: 0

0.5 Identify the Data to be Scraped

Write a couple of sentence on the data you want to scrape

Wir werden alle verfügbaren Teilnehmenden des Vienna City Marathon (nur Vollmarathon) über alle verfügbaren Jahre auf der MikaTiming-Plattform scrapen.

Aus Datenschutzgründen erfassen wir **keine Klarnamen**. Pro Person speichern wir nur Jahr, Event, Startnummer, Teilnehmer-ID und Ergebnis-Zeit.

Gescraped wird mit einem User-Agent, mit mindestens 1 Sekunde Pause zwischen Requests/Retrys und mit möglichst wenig unnötigem Traffic.

Auf der Website sind die Ergebnisse als HTML-Liste aufgebaut: Jedes -Element entspricht einem Teilnehmer und enthält u. a. Name, Startnummer und Zielzeit. Die Seite lädt diese Liste dynamisch über HTTP-GET-Requests mit URL-Parametern wie **event**, **page** und **num_results**. Wir nutzen denselben Mechanismus und laden die Seiten direkt mit **requests**, ohne einen Browser zu benötigen. Da pro Seite maximal 100 Einträge angezeigt werden, können wir pro Jahr bis zu 70 Seiten abrufen. Sobald eine Seite keine Einträge mehr liefert, wird die Schleife frühzeitig abgebrochen und mit dem nächsten Jahr fortgesetzt.

0.6 Extract Data

Find specific elements and extract text or attributes from elements (handle pagination if necessary)

Wir lesen jetzt alle Ergebnisseiten mit Pagination aus und sammeln die Laufdaten.

```

[15]: import time
import random
import re

# Event-IDs für den Vollmarathon je Jahr
EVENT_BY_YEAR = {
    2017: ("MAL", "Vienna City Marathon"),
    2018: ("MAL", "Vienna City Marathon"),
    2019: ("MAL", "Vienna City Marathon"),
    2020: ("MAL", "Vienna City Marathon"),
    2021: ("MAL_2EF80HST191", "Vienna City Marathon"),
    2022: ("MAL_HCH80HST1BZ", "Vienna City Marathon"),
    2023: ("MAL_HCH80HST1E1", "Vienna City Marathon"),
    2024: ("MAL_HCH80HST20A", "Vienna City Marathon"),
    2025: ("MAL_HCH80HST231", "Vienna City Marathon"),
    2026: ("MAL_2EF80HST259", "Vienna City Marathon"),
}

# Maximal 70 Seiten pro Jahr bei 100 Einträgen pro Seite
PAGES_BY_YEAR = {year: 99 for year in EVENT_BY_YEAR}

last_request_ts = 0.0
records = []

years_to_scrape = [y for y in range(START_YEAR, END_YEAR + 1) if y in EVENT_BY_YEAR]
print("Years to scrape:", years_to_scrape)

for year in years_to_scrape:
    event_id, event_name = EVENT_BY_YEAR[year]
    max_pages = PAGES_BY_YEAR.get(year, 99)

    year_records_before = len(records)
    pages_done = 0

    for page in range(1, max_pages + 1):
        params = {
            "page": page,
            "event": event_id,
            "num_results": 100,
            "pid": "list",
            "pidp": "start",
            "search[sex]": "%",
            "search[age_class]": "%",
        }

```

```

# Ethisches Scraping: mindestens 1s Abstand zwischen Anfragen plus
↳ vorsichtige Retries
response = None
last_error = None
for attempt in range(1, 4):
    wait = REQUEST_DELAY_SECONDS - (time.time() - last_request_ts)
    if wait > 0:
        time.sleep(wait)

    try:
        response = session.get(f"{BASE_DOMAIN}/{year}/", params=params,
↳ timeout=30)
        last_request_ts = time.time()

        if response.status_code == 200:
            break

        if response.status_code in (403, 429) and attempt < 3:
            time.sleep((2 * attempt) + random.uniform(0.1, 0.5))
            continue

        last_error = RuntimeError(f"HTTP {response.status_code}")
    except Exception as exc:
        last_error = exc
        if attempt < 3:
            time.sleep(attempt)

    if response is None or response.status_code != 200:
        raise RuntimeError(f"Request failed for year={year}, page={page}:
↳ {last_error}")

soup = BeautifulSoup(response.text, "html.parser")
items = soup.find_all("li", class_="list-group-item")

page_records = 0
if items:
    for item in items:
        if "list-group-header" in item.get("class", []):
            continue

        name_link = item.select_one("h4.type-fullname a")
        if not name_link:
            continue

        # Daten auslesen und in 'records' speichern
        href = name_link.get("href", "")
        id_match = re.search(r"[?&]idp=([^&]+)", href)

```

```

    participant_id = id_match.group(1) if id_match else None
    if not participant_id:
        continue

    bib_div = item.find("div", class_="field-start_no_text")
    bib_number = bib_div.get_text(strip=True) if bib_div else None

    run_time = None
    time_div = item.find("div", class_="type-time")
    if time_div:
        time_match = re.search(r"(\d{1,2}:\d{2}:\d{2})", time_div.
↪get_text(strip=True))
        if time_match:
            run_time = time_match.group(1)

    records.append({
        "year": year,
        "event_name": event_name,
        "bib_number": bib_number,
        "participant_id": participant_id,
        "run_time": run_time,
    })
    page_records += 1
else:
    # 2017 nutzt noch eine Tabellenansicht statt list-group-Elementen.
    rows = soup.select("table.list-table tbody tr")
    for row in rows:
        cells = row.find_all("td")
        if len(cells) < 7:
            continue

        name_link = cells[3].find("a", href=True)
        if not name_link:
            continue

        href = name_link.get("href", "")
        id_match = re.search(r"[?&]idp=(^[&]+)", href)
        participant_id = id_match.group(1) if id_match else None
        if not participant_id:
            continue

        bib_number = cells[2].get_text(strip=True) or None
        run_time_text = cells[6].get_text(strip=True)
        time_match = re.search(r"(\d{1,2}:\d{2}:\d{2})", run_time_text)
        run_time = time_match.group(1) if time_match else None

        records.append({

```

```

        "year": year,
        "event_name": event_name,
        "bib_number": bib_number,
        "participant_id": participant_id,
        "run_time": run_time,
    })
    page_records += 1

    pages_done += 1

    if page_records == 0:
        pages_done -= 1
        break

    year_total = len(records) - year_records_before
    print(f"{year} ({pages_done} Seiten): {year_total} Teilnehmer")

print(f"\nGesamt extrahierte Zeilen: {len(records)}")

```

```

Years to scrape: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026]
2017 (64 Seiten): 6340 Teilnehmer
2018 (55 Seiten): 5442 Teilnehmer
2019 (58 Seiten): 5733 Teilnehmer
2020 (0 Seiten): 0 Teilnehmer
2021 (31 Seiten): 3047 Teilnehmer
2022 (49 Seiten): 4803 Teilnehmer
2023 (67 Seiten): 6650 Teilnehmer
2024 (75 Seiten): 7490 Teilnehmer
2025 (94 Seiten): 9301 Teilnehmer
2026 (87 Seiten): 8668 Teilnehmer

```

Gesamt extrahierte Zeilen: 57474

0.7 Store Data in a Structured Format

Wir bauen ein DataFrame, entfernen Duplikate und zeigen einfache Statistiken.

```

[16]: df_results = pd.DataFrame(records)

# Datenschutz: eventuell vorhandene Namensspalten entfernen
pii_cols = ["name", "first_name", "last_name"]
df_results = df_results.drop(columns=[c for c in pii_cols if c in df_results.
    ↪columns], errors="ignore")

safe_cols = ["year", "event_name", "bib_number", "participant_id", "run_time"]
available_safe_cols = [c for c in safe_cols if c in df_results.columns]
if available_safe_cols:
    df_results = df_results[available_safe_cols]

```

```

if not df_results.empty and {"year", "participant_id"}.issubset(df_results.
↳columns):
    # Duplikate anhand eindeutiger ID pro Jahr entfernen
    df_results = df_results.drop_duplicates(subset=["year", "participant_id"])

print("DataFrame shape:", df_results.shape)
print("Columns:", list(df_results.columns))

if not df_results.empty:
    if "year" in df_results.columns:
        print("\nRows per year:")
        print(df_results.groupby("year").size())

        if "event_name" in df_results.columns:
            print("\nRows per event_name:")
            print(df_results.groupby("event_name").size().
↳sort_values(ascending=False))

df_results.head(10)

```

DataFrame shape: (57473, 5)

Columns: ['year', 'event_name', 'bib_number', 'participant_id', 'run_time']

Rows per year:

```

year
2017    6340
2018    5441
2019    5733
2021    3047
2022    4803
2023    6650
2024    7490
2025    9301
2026    8668
dtype: int64

```

Rows per event_name:

```

event_name
Vienna City Marathon    57473
dtype: int64

```

```

[16]:   year      event_name bib_number      participant_id  run_time
0  2017  Vienna City Marathon      M15  0000171A87617F00000A4291  02:08:40
1  2017  Vienna City Marathon      F3   0000171A87617F00000A426E  02:24:20
2  2017  Vienna City Marathon      M7   0000171A87617F00000A427D  02:08:42
3  2017  Vienna City Marathon      F4   0000171A87617F00000A4271  02:24:25

```

4	2017	Vienna City Marathon	M12	0000171A87617F00000A4289	02:09:10
5	2017	Vienna City Marathon	F6	0000171A87617F00000A4275	02:25:17
6	2017	Vienna City Marathon	M14	0000171A87617F00000A428F	02:10:24
7	2017	Vienna City Marathon	F1	0000171A87617F00000A4269	02:26:06
8	2017	Vienna City Marathon	M19	0000171A87617F00000A429A	02:10:51
9	2017	Vienna City Marathon	F10	0000171A87617F00000A4281	02:26:31

0.8 Save the Data

Wir speichern das bereinigte Ergebnis als CSV-Datei.

```
[17]: output_file = "vienna_city_marathon_all_years_participants.csv"
to_save = df_clean if "df_clean" in globals() else df_results
to_save.to_csv(output_file, index=False)
print(f"Saved {len(to_save)} rows to {output_file}")
```

Saved 53014 rows to vienna_city_marathon_all_years_participants.csv

0.9 Short Data Cleaning

Wir machen eine kurze Datenbereinigung auf nicht-personenbezogenen Feldern und zeigen die bereinigten Daten auszugsweise an.

```
[18]: # Reinigen
df_clean = (
    df_results
    .dropna(subset=["year", "event_name", "participant_id"])
    .drop_duplicates(subset=["year", "participant_id"])
    .copy()
)

required_cols = ["year", "event_name", "bib_number", "participant_id",
    ↪ "run_time"]
available_cols = [c for c in required_cols if c in df_clean.columns]
df_clean = df_clean[available_cols]

print("Cleaned shape:", df_clean.shape)
print("Missing run_time:", df_clean["run_time"].isna().sum() if "run_time" in
    ↪ df_clean.columns else "n/a")
print("Missing participant_id:", df_clean["participant_id"].isna().sum())

df_clean[["year", "event_name", "bib_number", "participant_id", "run_time"]].
    ↪ head(25)
```

Cleaned shape: (57473, 5)

Missing run_time: 10

Missing participant_id: 0


```

[18]:   year      event_name bib_number      participant_id run_time
0  2017  Vienna City Marathon      M15  0000171A87617F00000A4291  02:08:40
1  2017  Vienna City Marathon      F3   0000171A87617F00000A426E  02:24:20
2  2017  Vienna City Marathon      M7   0000171A87617F00000A427D  02:08:42
3  2017  Vienna City Marathon      F4   0000171A87617F00000A4271  02:24:25
4  2017  Vienna City Marathon     M12   0000171A87617F00000A4289  02:09:10
5  2017  Vienna City Marathon      F6   0000171A87617F00000A4275  02:25:17
6  2017  Vienna City Marathon     M14   0000171A87617F00000A428F  02:10:24
7  2017  Vienna City Marathon      F1   0000171A87617F00000A4269  02:26:06
8  2017  Vienna City Marathon     M19   0000171A87617F00000A429A  02:10:51
9  2017  Vienna City Marathon     F10   0000171A87617F00000A4281  02:26:31
10 2017  Vienna City Marathon      M3   0000171A87617F00000A4270  02:10:55
11 2017  Vienna City Marathon      F5   0000171A87617F00000A4272  02:29:25
12 2017  Vienna City Marathon     M18   0000171A87617F00000A4298  02:12:39
13 2017  Vienna City Marathon      F8   0000171A87617F00000A427B  02:34:33
14 2017  Vienna City Marathon     M17   0000171A87617F00000A4296  02:14:17
15 2017  Vienna City Marathon      F7   0000171A87617F00000A4278  02:36:31
16 2017  Vienna City Marathon     M26   0000171A87617F00000A42A5  02:14:50
17 2017  Vienna City Marathon     F11   0000171A87617F00000A4283  02:39:46
18 2017  Vienna City Marathon     M25   0000171A87617F00000A42A0  02:16:14
19 2017  Vienna City Marathon     F18   0000171A87617F00000A4290  02:46:03
20 2017  Vienna City Marathon     M20   0000171A87617F00000A429D  02:18:25
21 2017  Vienna City Marathon     F22   0000171A87617F00000A42B9  02:47:23
22 2017  Vienna City Marathon     M28   0000171A87617F00000A42AA  02:18:43
23 2017  Vienna City Marathon     F21   0000171A87617F00000A4299  02:48:48
24 2017  Vienna City Marathon     M11   0000171A87617F00000A4286  02:20:21

```